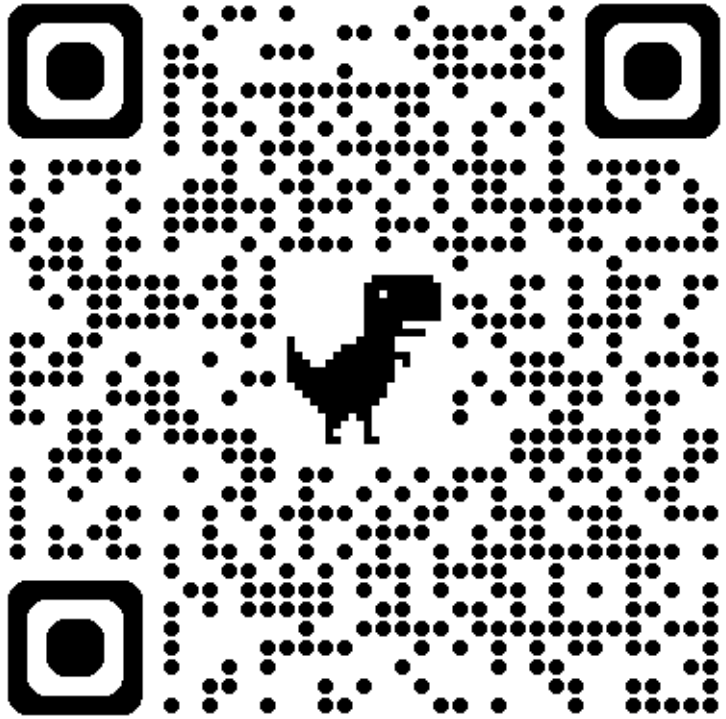
Kafka’s quiet observability superpower — Kafka Interceptors

J3, signalRoom // Brett Au, Life360



Agenda

01 ORIGIN

Genesis

Turning the Idea into Reality

02 BUILD

Building Kafka Isotope Tracing

Why Flink? Prometheus vs. Flink

03 HOW IT WORKS

Two Propagation Models

Out-of-Band Propagation

In-Band Propagation

04 PROOF → IMPACT → FUTURE

Isotope Headers & Ladder of Insights

Reference Order Event Pipeline

Life360: Problem, Metrics & Results

What's Next — AI!

Genesis

THE QUESTION THAT LED TO KAFKA ISOTOPE

1

GLOBAL ASSET MANAGER

“How do we capture Kafka client telemetry?”

The starting question

2

FIRST ANSWER

Kafka Interceptors + OTel

Capture client-level telemetry

3

LIFE360

“But how long does a message take to traverse the entire pipeline?”

The question that changed the scope

Client telemetry → end-to-end pipeline observability

Turning the Idea into Reality

FROM IDEA → OPEN SOURCE → PRODUCTION

1

PROPOSED

Isotope Tracing

A new way to observe end-to-end Kafka event journeys

2

COMMISSIONED

Life360 funded the open-source implementation

The idea became a reusable Kafka tracing library

3

INTEGRATED

Life360's Kafka Spring Client

The implementation moved into a real application platform

A tracing concept became deployable infrastructure.

Building Kafka Isotope Tracing

FROM APPLICATION SIGNALS TO PIPELINE INSIGHT

1

SOURCE

Kafka Java Applications

Business events enter the pipeline

2

CAPTURE

kafka-isotope library

Interceptors capture tracing signals

3

TRAVEL

Kafka event pipeline

The isotope travels with each event

4

INTERPRET

Apache Flink

Signals → insight

5

REPORT

7 Reports

End-to-end observability

THE PAYOFF



Do We Really Need Flink?

Prometheus is the right tool for *stateless metrics aggregation*.

Flink is the right tool when analysis requires *state, correlation, and event-time reasoning*.

Prometheus is very good at signaling **that something happened**. But `stuck_trace` requires us to **remember a trace, wait in event time, and determine that something we expected didn't happen**. That is why we use Flink—it is built for stateful stream processing.

Prometheus + Grafana	**Apache Flink**
Stateless metrics	Stateful stream processing
`latency`	`latency_percentiles`
`topology`	`coverage`
`hop_distribution`	`bipartite_topology`
	`stuck_trace`
Aggregate what happened	***Reason about what happened or didn't***

Behind the Curtain: Two Propagation Models

An **Isotope** is a ***lightweight tracing artifact attached to Kafka record headers***. Like a biochemical isotope used to trace molecules through a metabolic pathway, it lets you observe and analyze a record's journey through an event-driven architecture.

Kafka Isotope supports two propagation models for consume → produce processing:

- **OUT-OF-BAND PROPAGATION** — `IsotopeContext.adoptFromRecord()` places the inbound isotope into thread-local context. The next `send()` retrieves that context and propagates the trace.
- **IN-BAND PROPAGATION** — the isotope remains attached to the record as it moves through the processing graph. The producer retrieves the isotope from the outgoing record itself, so propagation survives thread, task, shuffle, and JVM boundaries.

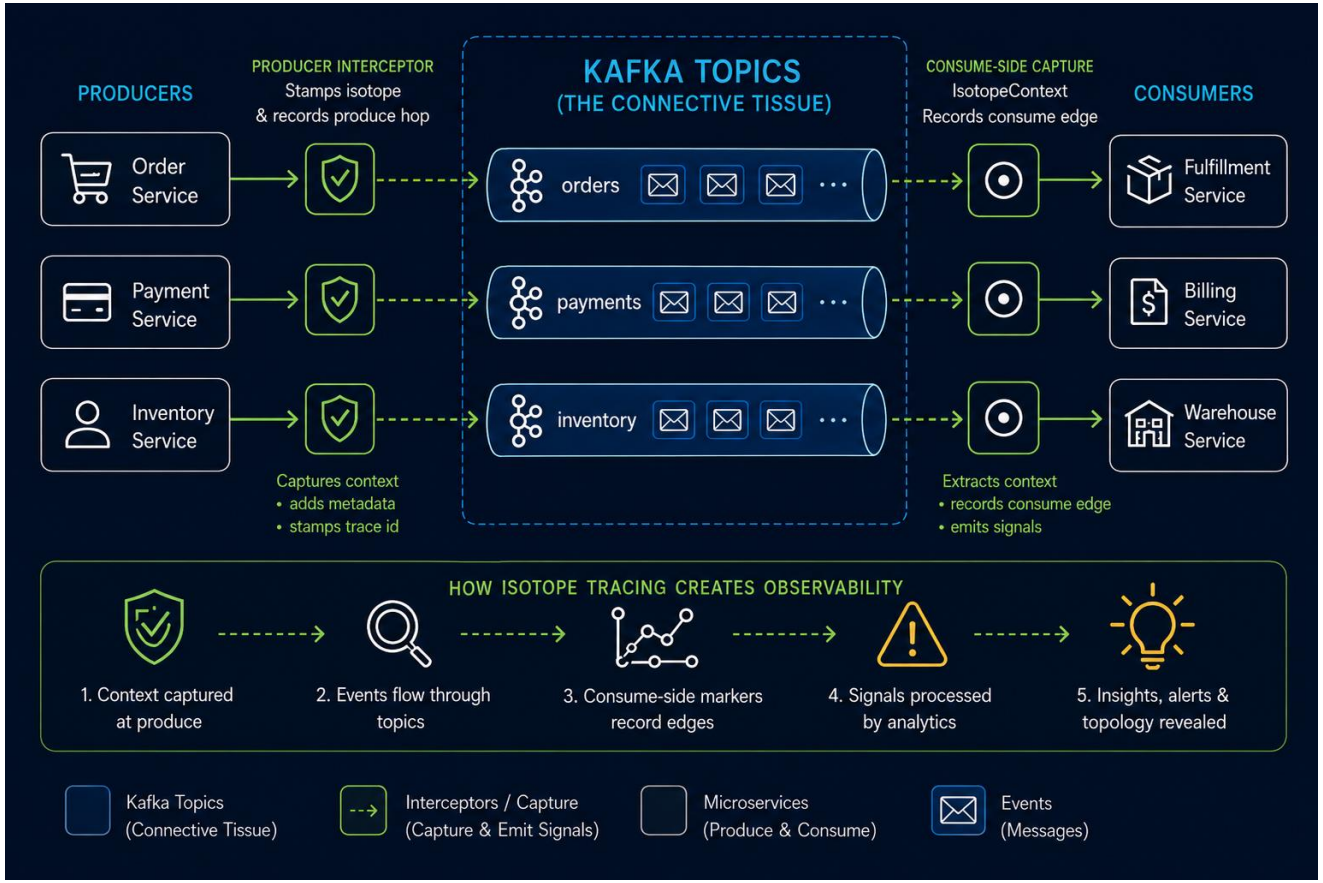
Same isotope. Same reports. Different propagation mechanism.

Out-Of-Band Propagation

```
consume → adoptFromRecord() →
IsotopeContext → produce
```

Best fit: Kafka microservices that consume, process, and produce within the same execution context.

The isotope itself still travels in the Kafka record header, but its **handoff between consume and produce occurs outside the business record**.

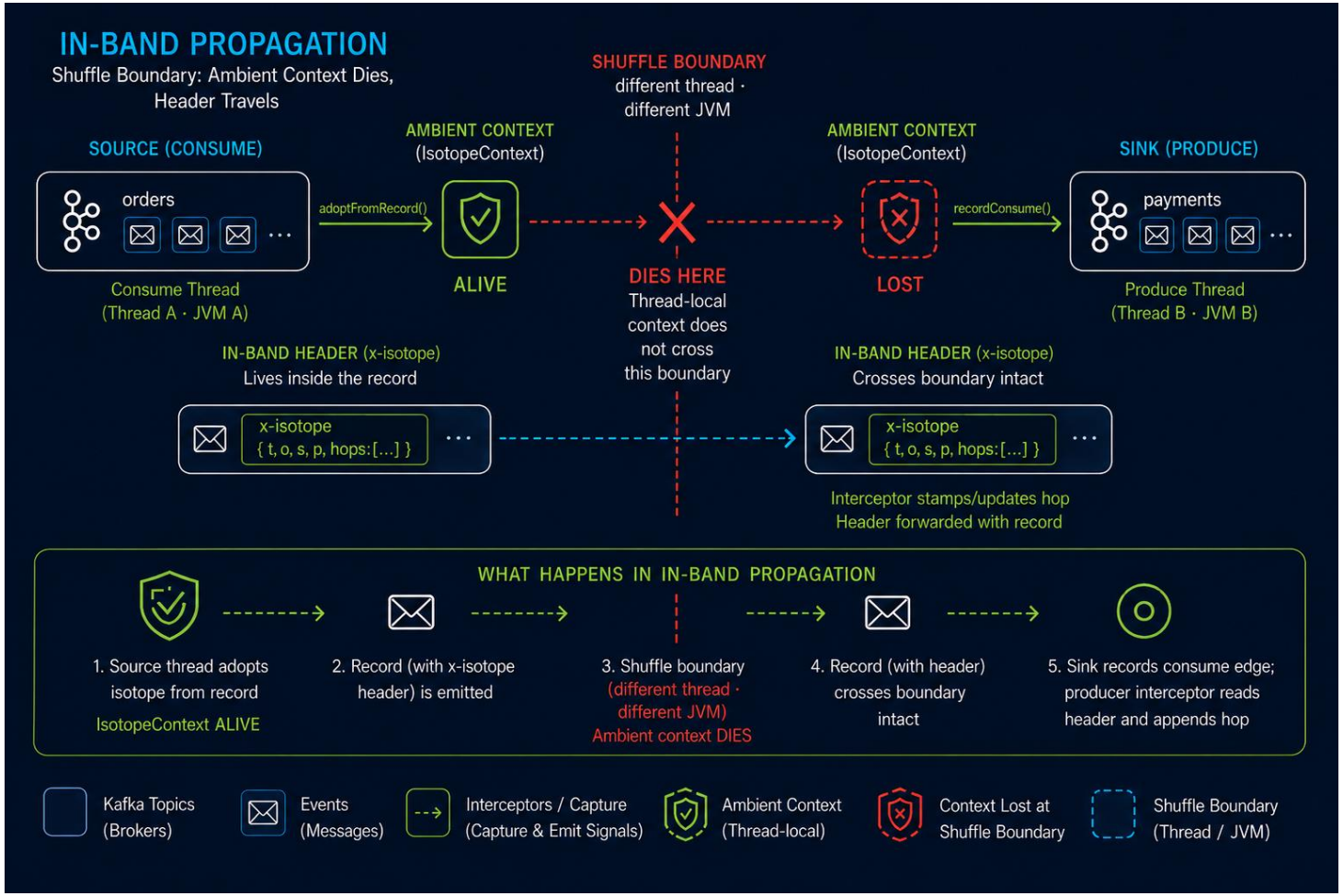


In-Band Propagation

```
source → record[isotope] →
shuffle → record[isotope] →
sink
```

Best fit: Apache Flink, where records cross threads, tasks, network shuffles, or JVMs.

The isotope **travels within the record header**, so it does not rely on thread-local state to survive the boundary.

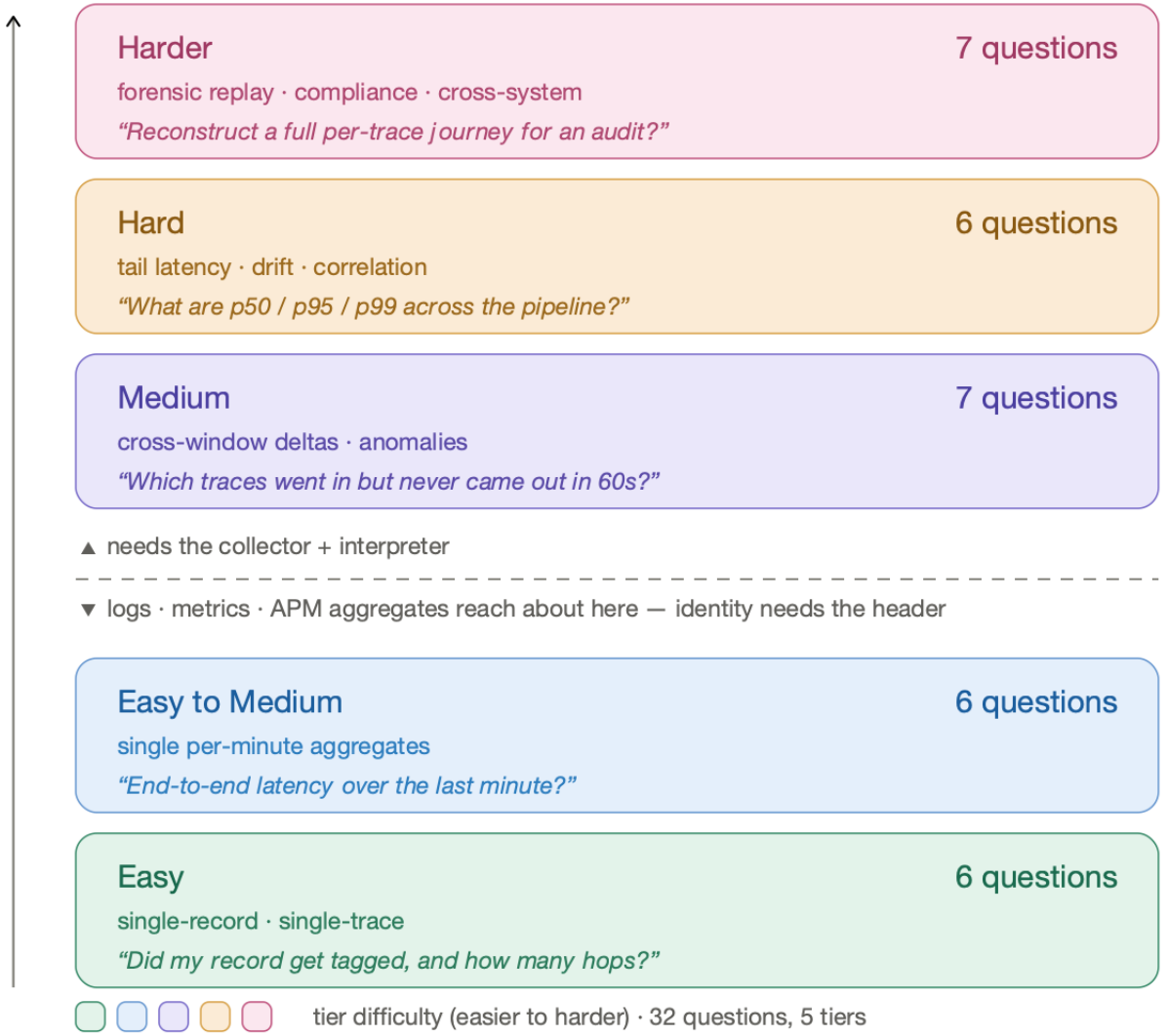


A Kafka Isotope in the Wild


```
{
        "x-isotope-hop-count": "1",
        "x-isotope":
"{\"t\":\"AZ69OS8OeG+9zufGfF2sbw==\",\"o\":1781291101966,\
"s\":\"order-intake-
service\",\"p\":\"order\",\"h\":[{\"s\":\"order-intake-
service\",\"t\":\"orders.placed\",\"m\":1781291101966}],\"
x\":false}",
        "x-isotope-trace-id":
"019ebd392f0e786fbdcee7c67c5dac6f",
        "x-isotope-this-service": "order-intake-service",
        "x-isotope-origin-service": "order-intake-service",
        "x-isotope-this-topic": "orders.placed",
        "x-isotope-origin-ts": "1781291101966",
        "x-isotope-pipeline": "order"
}
```

The Ladder of Insights

This end-to-end observability enabled by Isotope Tracing creates a ladder of insights, allowing developers to trace each event's journey, identify bottlenecks, and improve the performance and reliability of the entire system. The ladder is organized into **five tiers that answer 32 questions**.





Order Event Pipeline — CP on minikube

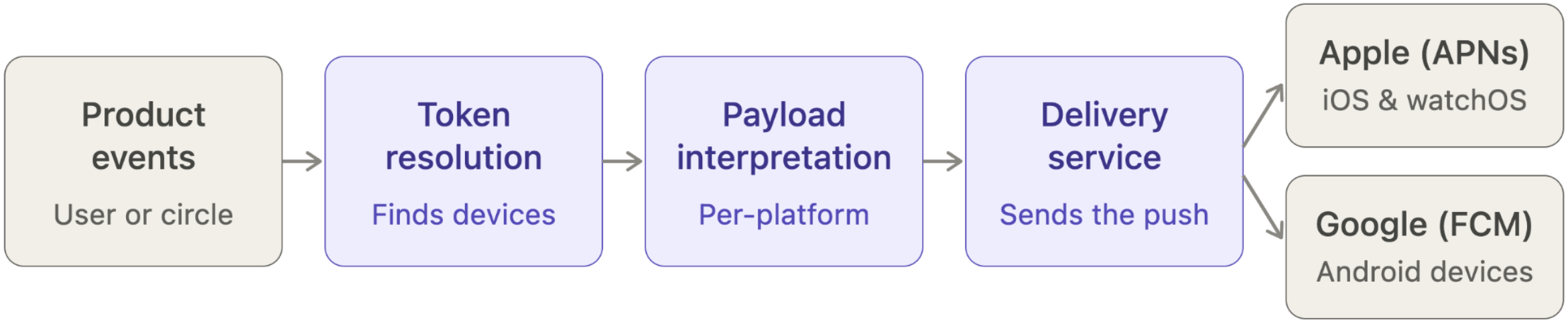
Life360

need and current situation

Common question from developers and product.
How long does a push notification take?



Architecture Diagram

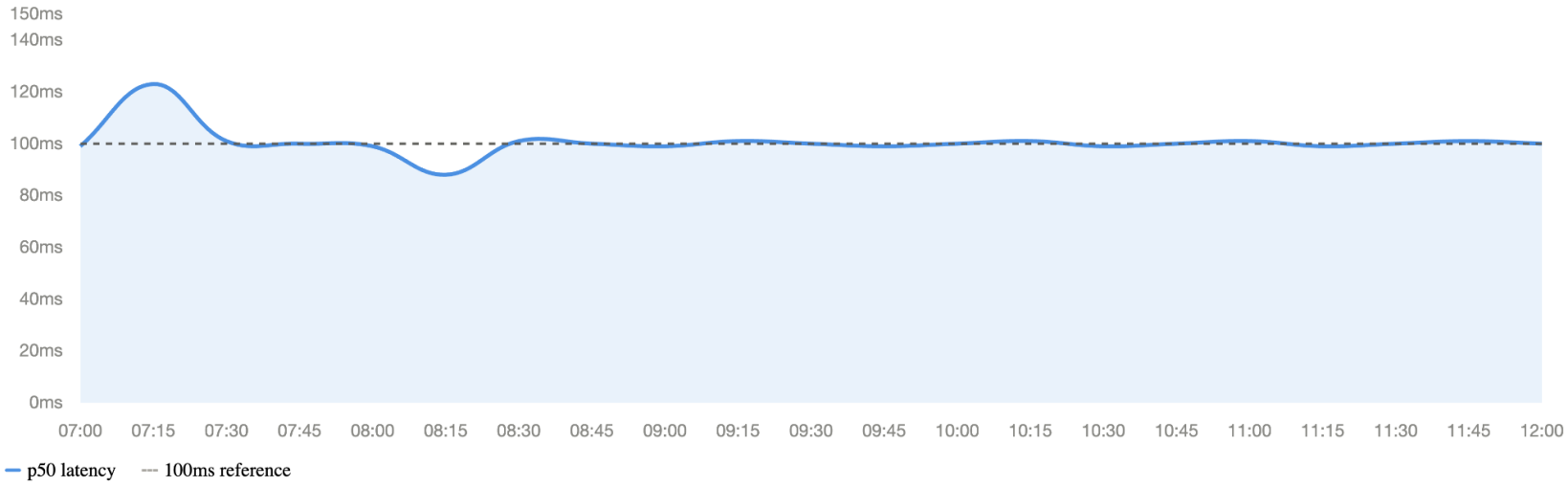


Token Service Producer Datadog Chart

Avg p50 latency
~100ms

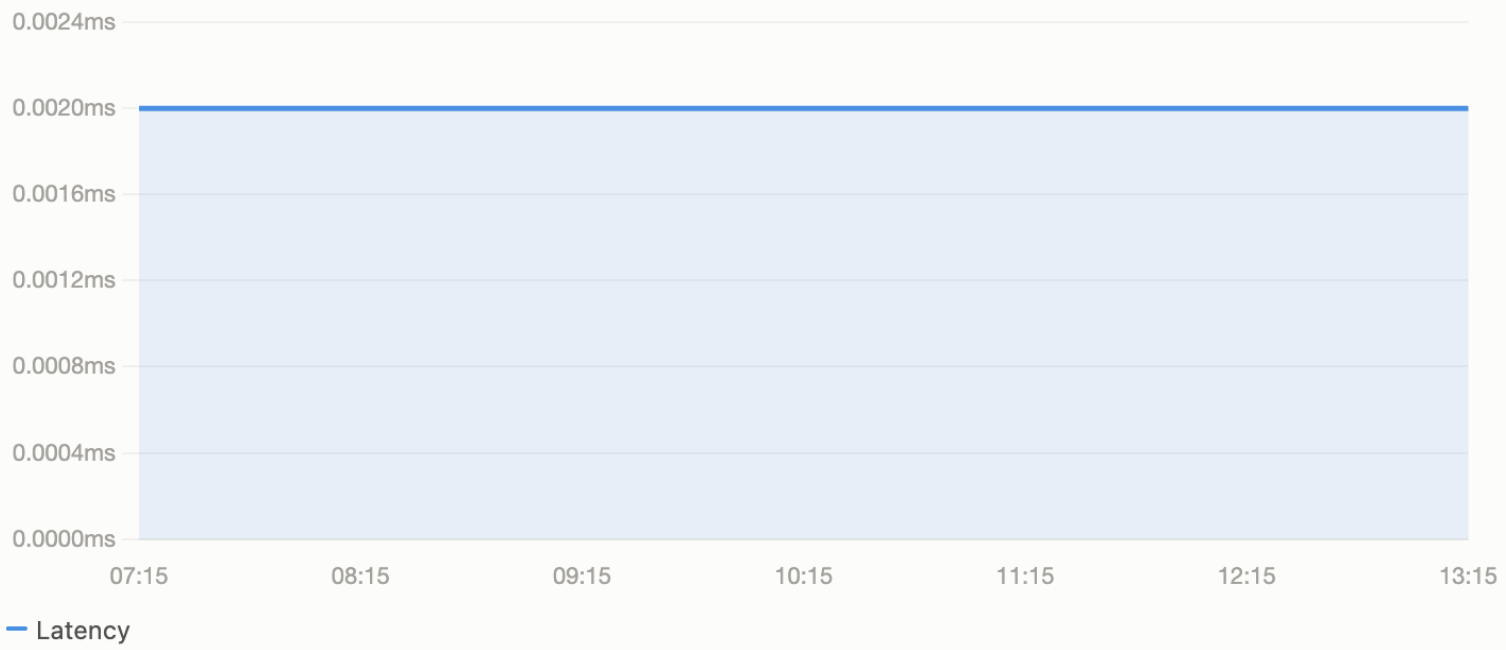
Peak observed
123ms

Window
5 hrs



Token Service Consumer Datadog Chart

Avg latency	Peak observed	Window
~0.002ms	0.002ms	6 hrs



How long does it take?

Consumer: <1ms

Producer: ~100ms

What does that mean? What is the p99 of a pipeline?



Our latency matches our Linger!

Shared library default

200ms

Tuned for fewer broker requests

Our service override

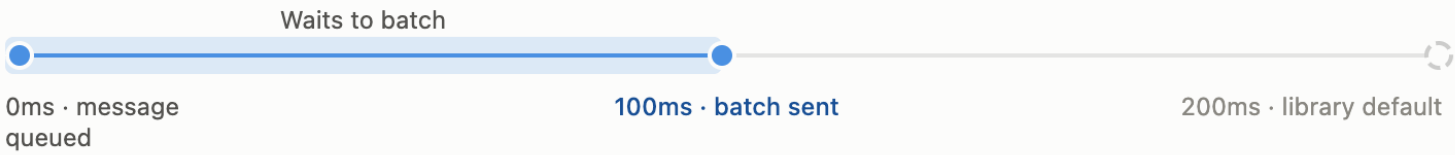
100ms

Tuned for push latency

application.yml

```yaml
spring:
  cloud:
    stream:
      kafka:
        binder:
          producer-properties:
            # Override the 200ms library default
            # for lower push latency
            linger.ms: 100
```

How a message leaves the producer



The Four Questions



How long do messages take to process?



Do we have outlier messages that process longer?



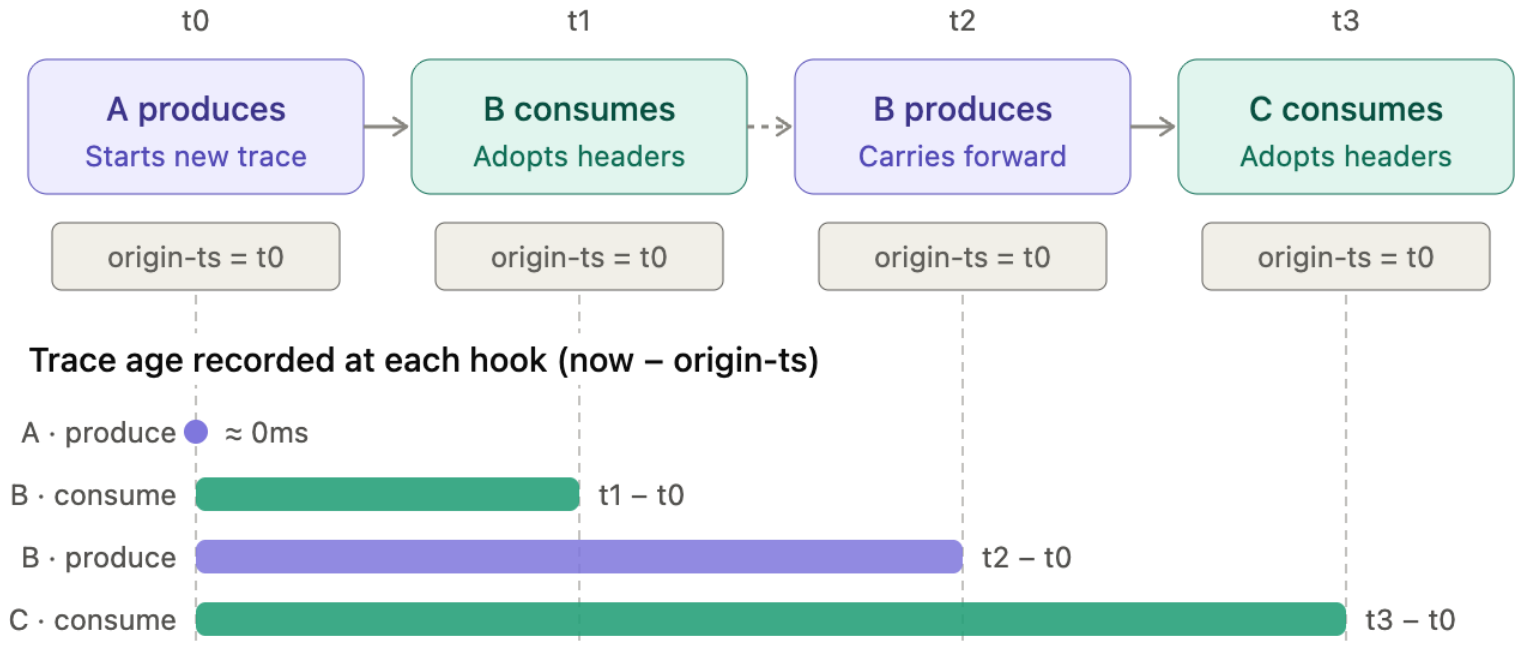
How long do messages sit in the local queue?



How can we see this information?



Isotope Metrics



Metric: isotope.trace.age {action, service, topic, pipeline, source}

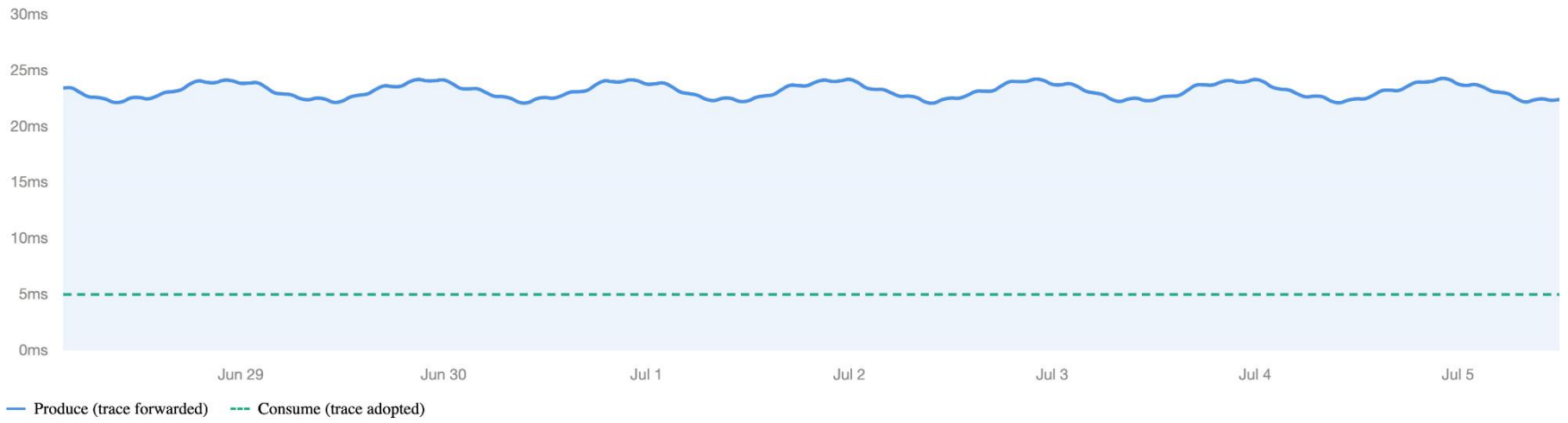
Per-hop latency = difference between consecutive bars

Token Service: Isotope Metrics

Consume p99
~5ms

Produce p99
22–24ms

Window
7 days



Look back at the Kafka Isotope Header

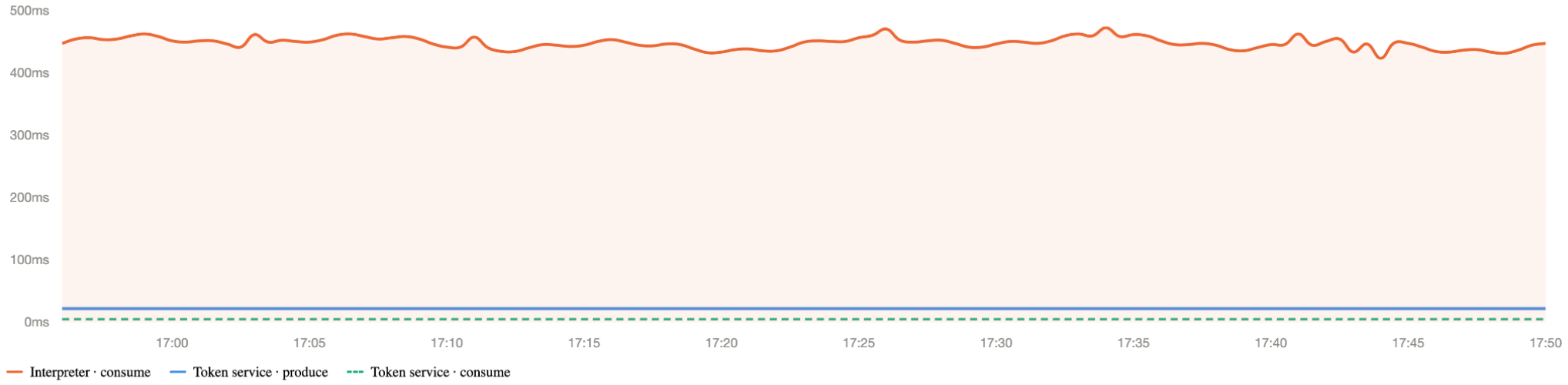

```
{
        "x-isotope-hop-count": "1",
        "x-isotope":
"{\"t\":\"AZ69OS8OeG+9zufGfF2sbw==\",\"o\":1781291101966,\
"s\":\"order-intake-
service\",\"p\":\"order\",\"h\":[{\"s\":\"order-intake-
service\",\"t\":\"orders.placed\",\"m\":1781291101966}],\"
x\":false}",
        "x-isotope-trace-id":
"019ebd392f0e786fbdcee7c67c5dac6f",
        "x-isotope-this-service": "order-intake-service",
        "x-isotope-origin-service": "order-intake-service",
        "x-isotope-this-topic": "orders.placed",
        "x-isotope-origin-ts": "1781291101966",
        "x-isotope-pipeline": "order"
}
```

Pipeline Metrics

Token service · consume
<5ms

Token service · produce
~22ms

Interpreter · consume
~450ms



Metrics — Part I

```
histogram_quantile(0.99, 3
  sum by (pipeline, action, le, app) ( 2
    rate(isotope_trace_age_seconds_bucket[2m]) 1
  )
) * 1000 4
```

1 Rate the buckets

Per-second growth of each latency bucket, smoothed over a 2-minute window.

2 Group the series

Merge pods into one line per service, hook, and pipeline. Keeps `le` so buckets stay intact.

3 Take the p99

Estimate the latency that 99% of messages beat, interpolated across the buckets.

4 Convert to ms

The timer records seconds; multiply by 1000 for milliseconds.

One line per `app` · which service `action` · consume or produce hook `pipeline` · message flow



Metrics — Part II

```
max by (pipeline) ( 5
  histogram_quantile(0.99, 3
    sum by (pipeline, action, le, app) ( 2
      rate(isotope_trace_age_seconds_bucket[2m]) 1
    )
  ) * 1000 4
)
```

1 Rate the buckets

Per-second growth of each latency bucket over 2 minutes.

2 Group the series

One line per service, hook, and pipeline. Keeps `le`.

3 Take the p99

Latency 99% of messages beat at that hook.

4 Convert to ms

Seconds × 1000.

5 Roll up per pipeline

Keep the slowest hook's p99 for each pipeline.

Why max works here: trace age only grows from the origin, so the slowest hook is the last one reached. Its p99 approximates the pipeline's end-to-end p99 — one line per pipeline.

What did we discover?

We had a 2.5-second p99 latency on our Push Notification Pipeline.

We found a serial processing service in our pipeline causing all of the delay.

We had no idea, and our latency in Datadog showed 100-200ms!



Isotope showed the improvement!

Before · via NSQ

2.5s

Frequent p99 spikes

After · via Kafka

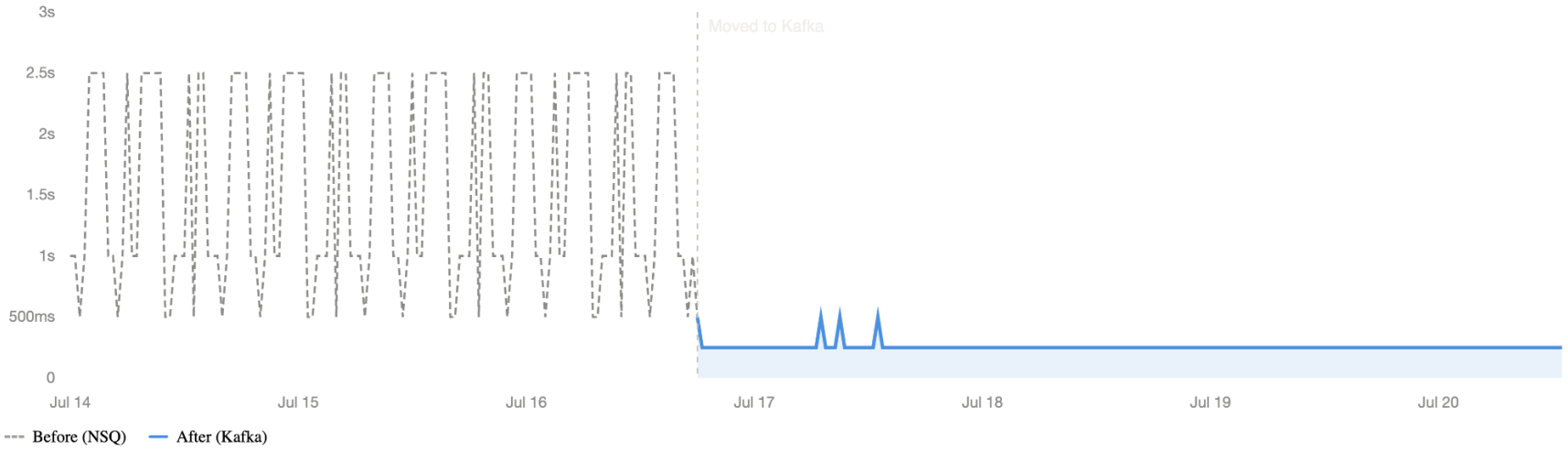
~250ms

Steady, rare blips

Improvement

10×

Lower worst-case p99



10×

lower p99 latency

Revealed by end-to-end trace headers after moving one hop from NSQ to Kafka

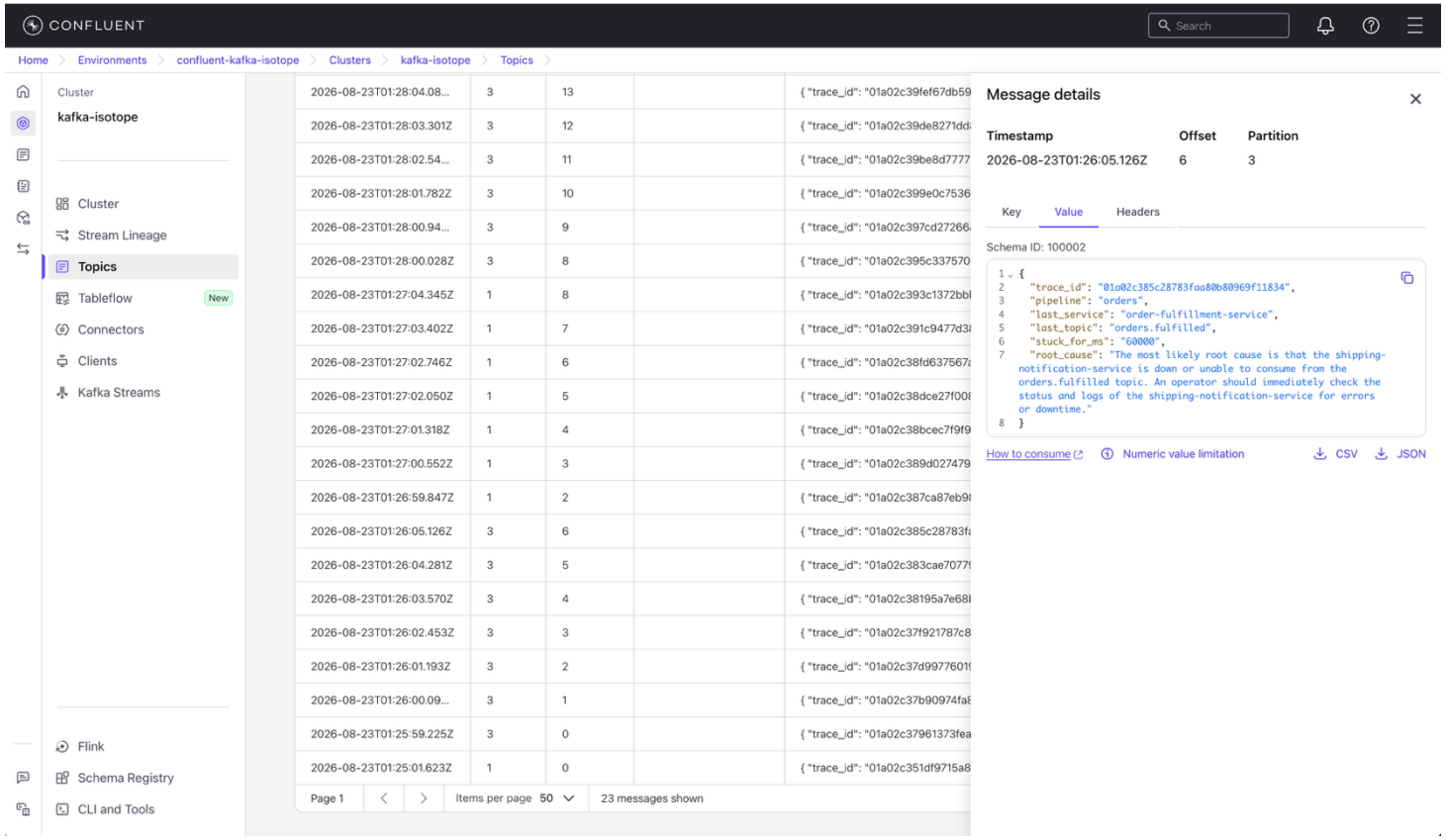


Time saved per message	Spikes after cutover	Code change to measure
~2.25s	Near zero	None

What's Next — AI!

Eighth Report — AI Root-Cause Analysis (RCA)

Beyond the seven deterministic reports, an **eighth, AI-generated report** uses Confluent Cloud for Apache Flink `CREATE MODEL` with OpenAI to turn each stuck-trace alert into a **natural-language root-cause hypothesis plus a one-line remediation**.



Thank you // Connect



